

ForensiCollect – Linux DFIR Triage Tool

Developed By: Caleb Clauson and Eric Anderson

Introduction

Digital forensic investigations are often time-consuming and require manually collecting and analyzing system data. The purpose of this project is to develop a tool that addresses the problem of analyzing and collecting critical system artifacts. Our project ForensiCollect solves this pain point by: Automating artifact collection; Structuring outputs for easy review; and generating a concise security summary. ForensiCollect is a Linux-based Digital Forensics and Incident Response (DFIR) triage tool designed to automate the collection and analysis of critical system artifacts. It enables cybersecurity analysts to quickly move from raw system data to actionable insights by combining automated evidence collection with AI-assisted reporting.

Technical Implementation

ForensiCollect was developed as a modular Linux digital forensics triage tool using Bash (Shell Scripting) as the primary programming language. Bash was chosen because it allows direct interaction with the operating system, making it highly effective for collecting low-level forensic artifacts such as logs, processes, and system configurations without requiring additional dependencies.

The tool follows a modular architecture, where each major category of forensic data is collected by a separate script located in the “modules/” directory. These modules include system information collection, user activity analysis, process and service enumeration, network analysis, and recent file change detection. This modular design improves maintainability, scalability, and readability, allowing new forensic capabilities to be easily added without modifying the core script.

The main controller script, “forensicollect.sh”, orchestrates the execution of all modules. It creates a structured case directory for each run, organizes outputs into raw and report subdirectories, logs execution steps, and ensures that all collected data is stored consistently. The script also generates a SHA-256 hash manifest to maintain data integrity, which is a critical requirement in digital forensics to ensure evidence has not been altered. For evidence collection, the tool leverages native Linux utilities such as, ps, top, and

systemctl for process and service analysis. Netstat and ss for network and port inspection, cat, grep and find for log parsing and file discovery. /var/log/auth.log and last for authentication activity, and lastly, crontab and /etc/cron.* for persistence detection. These tools were selected because they are standard across most Linux systems, ensuring compatibility and reliability without requiring additional installations.

To enhance analytical capabilities, ForensiCollect integrates an artificial intelligence component through an external API (SUSHI server). The AI module, implemented in “ai/ai_explainer.sh”, sends selected forensic artifacts to the AI model using HTTP POST requests via curl. The API request is structured in JSON format and includes a carefully engineered prompt designed to guide the AI in producing concise and relevant security analysis.

The AI integration allows the tool to move beyond simple data collection by automatically identifying suspicious behavior, such as unusual listening ports, potential reverse shells, or abnormal system activity. This significantly reduces the time required for manual analysis and helps users quickly understand the security posture of the system. Several techniques were implemented to ensure robust operation, including, input validation and error handling for missing files or logs, environment variable management using “.env” file for secure API key storage, output sanitization to ensure clean and readable AI-generated reports, logging of API responses for debugging and transparency, and a structured output generation in both human-readable (text) and machine-readable (JSON) formats.

Overall, the combination of Bash scripting, native Linux utilities, modular design, and AI integration enables ForensiCollect to efficiently automate the process of forensic data collection and initial analysis. This approach provides a lightweight yet powerful solution for rapid incident triage in Linux environments.

Results

When executed, ForensiCollect successfully collects and organizes systems artifacts into a structured case directory. The tool generates multiple outputs, including raw forensic data (logs, processes, and network information), a structured JSON report, a timeline of system activity, and an AI-generated summary designed to assist in rapid analysis.

Figure 1 shows the successful execution of the tool, including the sequential processing of each module and the completion of AI analysis. This confirms that all components of the system function as intended and designed.

```

vmuser@vmuser-Standard-PC-i440FX-PIIX-1996:~/IT_360_Final_Project_Spring_2026-main/src$ ./forensicollect.sh
[2026-04-26 12:42:46] ForensiCollect v1.0.0 started
[2026-04-26 12:42:46] CaseIdirectory: /home/vmuser/IT_360_Final_Project_Spring_2026-main/src/output/case_2026-04-26_124246
[2026-04-26 12:42:46] Raw directory: /home/vmuser/IT_360_Final_Project_Spring_2026-main/src/output/case_2026-04-26_124246/raw
[2026-04-26 12:42:46] Report directory: /home/vmuser/IT_360_Final_Project_Spring_2026-main/src/output/case_2026-04-26_124246/report
[2026-04-26 12:42:46] Checking dependencies...
[2026-04-26 12:42:46] Checking disk space...
[2026-04-26 12:42:46] Running module: system_info.sh
[2026-04-26 12:42:47] Completed module: system_info.sh
[2026-04-26 12:42:47] Running module: user_activity.sh
[2026-04-26 12:42:47] Completed module: user_activity.sh
[2026-04-26 12:42:47] Running module: process_service.sh
[2026-04-26 12:42:47] Completed module: process_service.sh
[2026-04-26 12:42:47] Running module: network.sh
[2026-04-26 12:42:47] Completed module: network.sh
[2026-04-26 12:42:47] Running module: recent_changes.sh
[2026-04-26 12:42:51] Completed module: recent_changes.sh
[2026-04-26 12:42:51] Collecting extended forensic artifacts...
[2026-04-26 12:42:52] Extended forensic artifact collection complete
[2026-04-26 12:42:52] Running AI analysis...
[2026-04-26 12:43:01] AI analysis completed
[2026-04-26 12:43:01] Writing summary...
[2026-04-26 12:43:01] Writing JSON report...
[2026-04-26 12:43:01] Hashing files...
[2026-04-26 12:43:01] Creating archive...
[2026-04-26 12:43:01] Collection complete
[2026-04-26 12:43:01] Output stored at: /home/vmuser/IT_360_Final_Project_Spring_2026-main/src/output/case_2026-04-26_124246
vmuser@vmuser-Standard-PC-i440FX-PIIX-1996:~/IT_360_Final_Project_Spring_2026-main/src$

```

Figure 1. Execution of './forensicollect' showing module processing

After execution, the tool creates a dedicated case directory containing all collected evidence and reports. As shown in Figure 2, the output structure includes raw data, processed reports, and integrity verification files such as hash manifests.

```

vmuser@vmuser-Standard-PC-i440FX-PIIX-1996:~/IT_360_Final_Project_Spring_2026-main/src$ ls output/case_2026-04-26_124246
collection_log.txt  raw      report.json  timeline.csv
hash_manifest.txt  report   summary.txt  warnings.txt

```

Figure 2. Generated case directory structure

The raw artifact collection (Figure 3) demonstrates the vast extent of data gathered by the tool. This includes authentication logs, network activity, user account data, cron jobs, and kernel modules. These artifacts provide a comprehensive view of system activity and are critical for identifying potential indicators of compromise.

```
vmuser@vmuser-Standard-PC-i440FX-PIIX-1996:~/IT_360_Final_Project_Spring_2026-main/src/output/case_2026-04-26_124246$ ls raw/
active_connections.txt  installed_packages.txt  recent_user_files.txt
arp.txt                ip_addr.txt             sudo_activity.txt
auth.log.txt           ip_route.txt            system_cron.txt
bash_history.txt       kernel_modules.txt      systemctl_services.txt
cron_dirs.txt          last_50.txt             top_snapshot.txt
date.txt               listening_ports.txt     uid0_users.txt
df.txt                 lsblk.txt               uname.txt
environment.txt        os-release.txt          uptime.txt
etc_recent_changes.txt passwd_full.txt          user_cron.txt
hostname.txt           passwd.txt               varlog_recent_changes.txt
important_users.txt    ps_aux.txt              who.txt
vmuser@vmuser-Standard-PC-i440FX-PIIX-1996:~/IT_360_Final_Project_Spring_2026-main/src/output/case_2026-04-26_124246$
```

Figure 3. Collected forensic artifacts

The AI-generated summary (Figures 4-9) provides an automated analysis of the collected data. The system successfully identified that most system activity was normal, including expected cron job behavior and standard system process. However, it also flagged a potentially suspicious network condition involving port 4444. Specifically, the tool detected that port 4444 was actively listening and associated with a netcat (nc) process. This is significant because port 4444 is commonly used for reverse shells or unauthorized remote access. While this activity may be part of a controlled test environment, it represents a realistic example of behavior that would require further investigation in a real-world scenario.

```
vmuser@vmuser-Standard-PC-i440FX-PIIX-1996:~/IT_360_Final_Project_Spring_2026-main/src/output/case_2026-04-26_124246$ cat summary.txt
ForensiCollect Summary
=====
Version: 1.0.0
Case: /home/vmuser/IT_360_Final_Project_Spring_2026-main/src/output/case_2026-04-26_124246
Collection Time: 2026-04-26 12:43:01
Warnings: 0

Artifacts:
- raw/ contains collected system evidence
- report/ contains AI and analysis outputs
- collection_log.txt contains the audit trail
- hash_manifest.txt contains SHA-256 hashes
- timeline.csv contains collection timeline events

Core Modules:
- system_info.sh
- user_activity.sh
- process_service.sh
- network.sh
- recent_changes.sh

Extended Artifacts Added:
- sudo_activity.txt
- passwd_full.txt
- important_users.txt
- uid0_users.txt
- user_cron.txt
- system_cron.txt
- cron_dirs.txt
- active_connections.txt
- kernel_modules.txt
- recent_user_files.txt
- bash_history.txt
- environment.txt
- installed_packages.txt

AI Summary:
Generated: ai_summary.json

AI Summary Preview:
-----
{
  "tool": "ForensiCollect",
  "analysis_type": "AI Summary",
  "sections": {
    "host_info": {
      "hostname": " Static hostname: vmuser-Standard-PC-i440FX-PIIX-1996",
      "os": "Ubuntu 24.04.3 LTS"
    },
  },
}
```

```

"authentication_activity": {
  "failed_login_attempts": 0,
  "ai_analysis": "These log entries indicate that the cron daemon is running and opening/closing sessions for the root user every minute. This is a normal and expected behavior for the cron daemon, which runs system-wide cron jobs. There is no suspicious activity in these log entries. The cron daemon is simply running as expected, opening and closing sessions for the root user on a regular schedule."
},
"network_activity": {
  "listening_ports_detected": 11,
  "ai_security_assessment": "The following ports appear unusual or suspicious: * 4444: This is a commonly used port for backdoors or malicious activity. The fact that it's listening and has a process associated with it (nc) is unusual, as nc is typically used for network scanning and not for listening on a port. The other ports (5353, 53, 34390) appear to be legitimate and are associated with Avahi and systemd-resolved services, which are common on Linux systems."
},
"recent_system_changes": {
  "etc_changes": "2026-04-26 12:02:02.5973596850 /etc/cups/subscriptions.conf 2026-04-26 12:00:39.6447761000 /etc/cups/subscriptions.conf.0 ",
  "varlog_changes": "2026-04-26 12:42:47.4007313470 /var/log/journal/8bf07b116392477cab433992446c1405/system.journal 2026-04-26 12:42:47.0657335370 /var/log/syslog 2026-04-26 12:42:19.4009141440 /var/log/journal/8bf07b116392477cab433992446c1405/user-1000.journal 2026-04-26 12:42:19.0559163950 /var/log/auth.log 2026-04-26 12:40:24.4776615310 /var/log/sysstat/sa26 "
},
"process_snapshot": {
  "ai_process_analysis": "The following processes appear suspicious or unusual: * PID 4: '[kworker/R-rcu_gp]' - This process is running in idle state ('I<') with 0 CPU and memory usage, which is unusual for a kernel worker process. * PID 5: '[kworker/R-sync_wq]' - Similar to PID 4, this process is also running in idle state with 0 CPU and memory usage. * PID 6: '[kworker/R-kvfree_rcu_reclaim]' - Another idle kernel worker process with 0 CPU and memory usage. * PID 7: '[kworker/R-slub_flushwq]' - Another idle kernel worker process. * PID 8: '[kworker/R-netns]' - Another idle kernel worker process. * PID 11: '[kworker/0:0H-events_highpri]' - Another idle kernel worker process. These processes are all running in idle state, which is unusual for kernel worker processes. They also have 0 CPU and memory usage, which suggests that they are not actually doing any work. This could be a sign of a kernel bug or a malicious process trying to hide its activity."
},
"process_snapshot": {
  "ai_process_analysis": "The following processes appear suspicious or unusual: * PID 4: '[kworker/R-rcu_gp]' - This process is running in idle state ('I<') with 0 CPU and memory usage, which is unusual for a kernel worker process. * PID 5: '[kworker/R-sync_wq]' - Similar to PID 4, this process is also running in idle state with 0 CPU and memory usage. * PID 6: '[kworker/R-kvfree_rcu_reclaim]' - Another idle kernel worker process with 0 CPU and memory usage. * PID 7: '[kworker/R-slub_flushwq]' - Another idle kernel worker process. * PID 8: '[kworker/R-netns]' - Another idle kernel worker process. * PID 11: '[kworker/0:0H-events_highpri]' - Another idle kernel worker process. These processes are all running in idle state, which is unusual for kernel worker processes. They also have 0 CPU and memory usage, which suggests that they are not actually doing any work. This could be a sign of a kernel bug or a malicious process trying to hide its activity."
}
}

Warnings:
None

```



```

vmuser@vmuser-Standard-PC-i440FX-PIIX-1996:~/IT_360_Final_Project_Spring_2026-main/src/output/case_2026-04-26_124246/report$ cat ai_summary.json
{
  "tool": "ForensiCollect",
  "analysis_type": "AI Summary",
  "sections": {
    "host_info": {
      "hostname": " Static hostname: vmuser-Standard-PC-i440FX-PIIX-1996",
      "os": "Ubuntu 24.04.3 LTS"
    },
    "authentication_activity": {
      "failed_login_attempts": 0,
      "ai_analysis": "These log entries indicate that the cron daemon is running and opening/closing sessions for the root user every minute. This is a normal and expected behavior for the cron daemon, which runs system-wide cron jobs. There is no suspicious activity in these log entries. The cron daemon is simply running as expected, opening and closing sessions for the root user on a regular schedule."
    },
    "network_activity": {
      "listening_ports_detected": 11,
      "ai_security_assessment": "The following ports appear unusual or suspicious: * 4444: This is a commonly used port for backdoors or malicious activity. The fact that it's listening and has a process associated with it (nc) is unusual, as nc is typically used for network scanning and not for listening on a port. The other ports (5353, 53, 34390) appear to be legitimate and are associated with Avahi and systemd-resolved services, which are common on Linux systems."
    },
    "recent_system_changes": {
      "etc_changes": "2026-04-26 12:02:02.5973596850 /etc/cups/subscriptions.conf 2026-04-26 12:00:39.6447761000 /etc/cups/subscriptions.conf.0 ",
      "varlog_changes": "2026-04-26 12:42:47.4007313470 /var/log/journal/8bf07b116392477cab433992446c1405/system.journal 2026-04-26 12:42:47.0657335370 /var/log/syslog 2026-04-26 12:42:19.4009141440 /var/log/journal/8bf07b116392477cab433992446c1405/user-1000.journal 2026-04-26 12:42:19.0559163950 /var/log/auth.log 2026-04-26 12:40:24.4776615310 /var/log/sysstat/sa26 "
    },
    "process_snapshot": {
      "ai_process_analysis": "The following processes appear suspicious or unusual: * PID 4: '[kworker/R-rcu_gp]' - This process is running in idle state ('I<') with 0 CPU and memory usage, which is unusual for a kernel worker process. * PID 5: '[kworker/R-sync_wq]' - Similar to PID 4, this process is also running in idle state with 0 CPU and memory usage. * PID 6: '[kworker/R-kvfree_rcu_reclaim]' - Another idle kernel worker process with 0 CPU and memory usage. * PID 7: '[kworker/R-slub_flushwq]' - Another idle kernel worker process. * PID 8: '[kworker/R-netns]' - Another idle kernel worker process. * PID 11: '[kworker/0:0H-events_highpri]' - Another idle kernel worker process. These processes are all running in idle state, which is unusual for kernel worker processes. They also have 0 CPU and memory usage, which suggests that they are not actually doing any work. This could be a sign of a kernel bug or a malicious process trying to hide its activity."
    }
  }
}
vmuser@vmuser-Standard-PC-i440FX-PIIX-1996:~/IT_360_Final_Project_Spring_2026-main/src/output/case_2026-04-26_124246/report$

```

Figures 4-9. AI-generated forensic analysis summary

Figure 10 further supports this finding by displaying active listening ports on the system. The presence of a netcat process bound to port 4444 stands out compared to other normal system services, such as DNS and mDNS, which are expected in a standard Linux environment.

```
vmuser@vmuser-Standard-PC-i440FX-PIIX-1996:~/IT_360_Final_Project_Spring_2026-main$ ss -tulpen
```

Netid	State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port
Process					
udp	UNCONN	0	0	0.0.0.0:48304	0.0.0.0:*
uid:108 ino:10443 sk:1 cgroup:/system.slice/avahi-daemon.service <->					
udp	UNCONN	0	0	0.0.0.0:5353	0.0.0.0:*
uid:108 ino:10441 sk:2 cgroup:/system.slice/avahi-daemon.service <->					
udp	UNCONN	0	0	127.0.0.54:53	0.0.0.0:*
uid:991 ino:5831 sk:3 cgroup:/system.slice/systemd-resolved.service <->					
udp	UNCONN	0	0	127.0.0.53%lo:53	0.0.0.0:*
uid:991 ino:5829 sk:4 cgroup:/system.slice/systemd-resolved.service <->					
udp	UNCONN	0	0	:::5353	:::*
uid:108 ino:10442 sk:5 cgroup:/system.slice/avahi-daemon.service v6only:1 <->					
udp	UNCONN	0	0	:::34390	:::*
uid:108 ino:10444 sk:6 cgroup:/system.slice/avahi-daemon.service v6only:1 <->					
tcp	LISTEN	0	4096	127.0.0.53%lo:53	0.0.0.0:*
uid:991 ino:5830 sk:7 cgroup:/system.slice/systemd-resolved.service <->					
tcp	LISTEN	0	4096	127.0.0.54:53	0.0.0.0:*
uid:991 ino:5832 sk:8 cgroup:/system.slice/systemd-resolved.service <->					
tcp	LISTEN	0	1	0.0.0.0:4444	0.0.0.0:*
users:(("nc",pid=64287,fd=3)) uid:1000 ino:314757 sk:d cgroup:/user.slice/user-1000.slice/user@1000.service/app.slice/a					
pp-org.gnome.Terminal.slice/vte-spawn-5e47da00-0ad0-426c-864d-2a8523525d31.scope <->					
tcp	LISTEN	0	4096	127.0.0.1:631	0.0.0.0:*
ino:595985 sk:14 cgroup:/system.slice/cups.service <->					
tcp	LISTEN	0	4096	:::1:631	:::*
ino:595984 sk:15 cgroup:/system.slice/cups.service v6only:1 <->					

Figure 10. Example of detected suspicious network activity

Overall, the results demonstrate that ForensiCollect is capable of not only gathering relevant forensic data but also highlighting meaningful security insights. The combination of automated evidence collection and AI-assisted analysis significantly improves the speed and clarity of initial incident triage.

Lessons Learned & Conclusion

ForensiCollect demonstrates how automation and AI can enhance digital forensics through reducing investigation times, improving clarity of findings, and providing structured, repeatable analysis. After developing this tool over the course of a semester, many different lessons were learned. During the development of the tool, two of the main difficulties were debugging our scripts, as well as utilizing artificial intelligence from our SUSHI server API key. With the nature of our tool, debugging took a while. Since it is a Shell script, changes were made incrementally, as the project needed to be run after each change. With the AI integration, we faced many different challenges such as connecting with the API key, calling the API methods, as well as prompting and structuring the AI to make sure our reports are understandable and presentable for everyone.

The successful completion of this tool is a testament to our team's communication and work ethics, which allowed us to develop this project in a semester's time. This helped us prevail over any difficulties faced during the development process of this project. In the future, our team would like to make our reports more realistic and useful for different types of investigations. Additionally, we would like to have the project integrated into a web-facing

application that creates an easier to understand user interface, instead of a pure command line tool. Ultimately, ForensiCollect serves as a tool to modernize DFIR workflows. By synthesizing automation with Artificial Intelligence, our team has created a scalable framework that enhances both the speed and clarity of digital forensic reporting.